

Text-to-Image Generation using CLIP and Diffusion Models - SSY340 - Project Group 24

Daniel González Muela

Sotiris Koutsoftas

Abstract—This project explores the generation of high-quality images using diffusion models, while building a comprehensive understanding of the processes involved in text-to-image synthesis. The work is structured into two main parts. The first part focuses on the foundational steps of diffusion models, specifically Denoising Diffusion Probabilistic Models (DDPM) and Denoising Diffusion Implicit Models (DDIM), highlighting their application in generating high-quality outputs. Initial experiments include fine-tuning models on diverse datasets, such as celebrity faces and butterflies, demonstrating the capabilities of these models in adapting to different image domains. The second part extends these foundations by exploring advanced guidance techniques and bigger pre-trained diffusion models, while incorporating CLIP and BERT-based conditioning, to enhance the fidelity and alignment of generated images to specific prompts. This approach provides insights into the interplay between text and image embeddings and their impact on generative quality. The report concludes with a comparison of guidance mechanisms and their effectiveness in creating diverse, visually coherent outputs.

I. INTRODUCTION TO DIFFUSION MODELS

A. Traditional Generative Models

Traditional generative models, such as Variational Autoencoders (VAEs) and autoregressive models, have been widely used to model data distributions. However, they each come with inherent challenges [1], [2].

- VAEs operate by encoding data into a lower-dimensional latent space and then decoding it back, with a probabilistic framework that allows for the generation of new samples. While VAEs offer stability in training, they often struggle with generating sharp, high-quality images due to the smoothing effect of their probabilistic sampling.
- Autoregressive models generate data sequentially, predicting each element conditioned on the previous ones. These models can capture complex dependencies and generate high-quality samples but are often limited by slow sampling speeds due to their step-by-step nature.

Diffusion models address some of these challenges by providing a more stable training process without adversarial components. They can generate high-quality images by leveraging the understanding of data transformations through noise, making them a promising alternative to traditional methods.

B. Diffusion Models Overview

Diffusion models have gained attention for their ability to generate high-quality samples by learning to reverse a noising process. At the core of these models are two primary processes: forward diffusion and reverse diffusion [3].

- The forward diffusion process involves gradually adding noise to data, effectively transforming a structured input into pure noise. This transformation allows the model to learn how data deteriorates under noise, facilitating its understanding of how to reverse this process.
- Conversely, the reverse diffusion process works to recover the original data from the noisy distribution. By learning how to denoise or revert the noising process, the model can generate realistic samples starting from random noise.

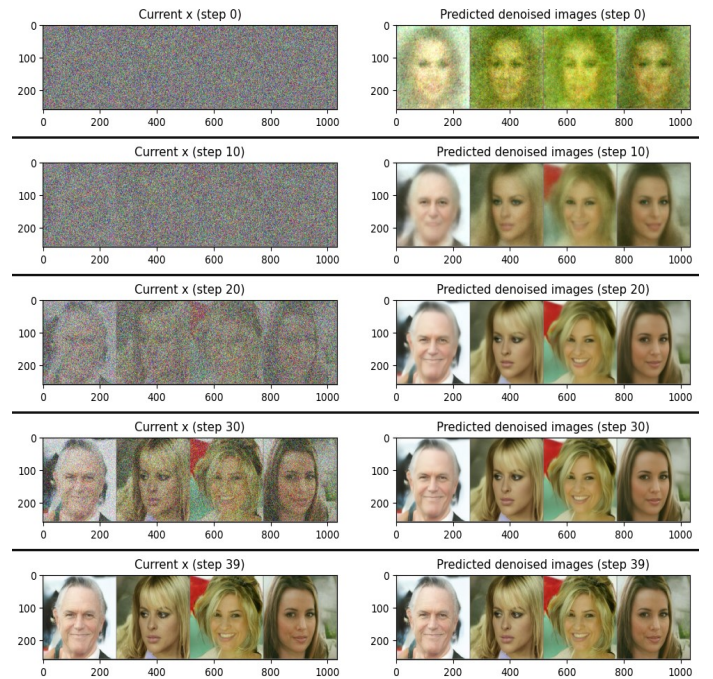


Fig. 1: Denoising process using the DDIM Scheduler over multiple steps. The figure shows the transformation from noisy input (left) to the denoised image (right) as steps progress.

II. DDPM AND DDIM

Denoising Diffusion Probabilistic Models (DDPM) [4] is one of the simplest types of diffusion models, widely recognized for its effectiveness in generating high-quality images. DDPM introduces a forward process where noise is added to images until they become random noise. The model is trained to reverse this process, progressively denoising random noise step-by-step. In contrast, Denoising Diffusion Implicit Models (DDIM) [5] offers an approach that allows for the generation of images with similar quality to DDPM but with fewer

iterations. DDIM estimates what the final image would look like based on fewer but larger steps, enhancing efficiency.

A. Initial Model Implementation

We begin our project by importing a pretrained DDPM model on a face dataset of high-quality celebrity images. While it generates very high-quality images, it takes around 2 minutes to generate a single image using DDPM due to its high number of iterations. Implementing a DDIM Scheduler substantially improved the generation time to around 20 seconds, as DDIM only requires 40 steps to achieve comparable quality.



B. Fine-Tuning on Different Datasets

Fine-tuning a model with new training data begins with a pre-trained model, which provides the advantage of leveraging weights that already represent useful features, rather than starting from scratch. We fine-tuned the model using a butterfly dataset [6] and observed the evolution of the generated images over several epochs. Initially, the images appeared noisy and unrecognizable (Figure 2), but as training progressed, Figure 3 shows how distinctive butterfly patterns started to emerge.

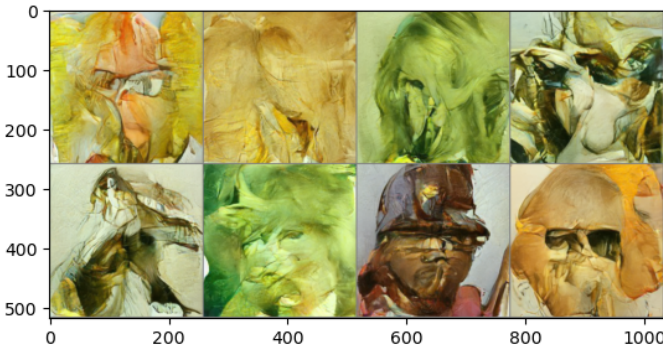


Fig. 2: Butterflies after 1 Epoch

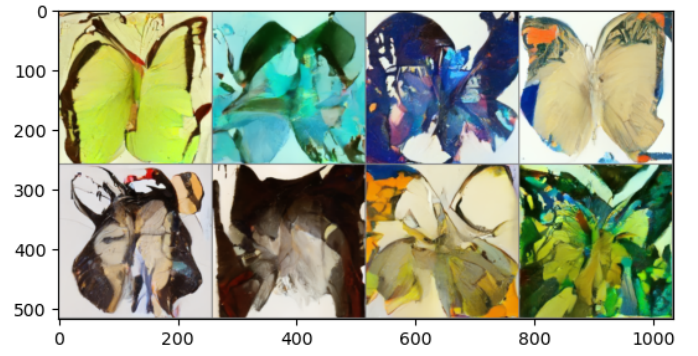


Fig. 3: Butterflies after 10 Epochs

C. Guidance Techniques

To explore guidance techniques, we applied them to a more diverse dataset, specifically using a DDPM model trained on a subset of bedrooms from the WikiArt dataset [7]. By defining a loss function that compares the pixels of an image to a target color, we guided the diffusion process to produce images closer to the specified condition, as shown in Figure 5.



Fig. 4: Original Room Image



Fig. 5: Guided Room Image with Green Color Condition

D. Zero-shot Guidance with CLIP

CLIP is a model designed to align text and image representations within a shared latent space, which will be explained in more detail in the next section. We leverage this capability by conditioning our diffusion models on CLIP's text embeddings as a guidance mechanism. However, this technique does not always generate images that closely match textual prompts.

For instance, when conditioning a fine-tuned model trained on butterfly data using a CLIP loss with the prompt "Red Rose (still life), red flower painting," the generated outputs displayed on Figures 6 and 7 often remain reminiscent of the original images the model was trained on, rather than fully aligning with the new text description. This could be because the underlying models are overfitted to small subsets of images, making it difficult to generalize. Additionally, since CLIP operates in a latent space and DDPMs work in pixel space, aligning text and images becomes more challenging and computationally intensive.



Fig. 6: CLIP Guidance applied to the WikiArt Dataset



Fig. 7: CLIP Guidance applied to the Butterfly Dataset

In conclusion, DDPM is one of the most basic diffusion models, and its performance can be substantially improved using a DDIM scheduler. We demonstrated that both models are capable of generating realistic results after training, and that fine-tuning can achieve similar outcomes to training from scratch. One approach to generalizing the model for new image generation is through guidance. While guiding through a loss function based on color works well, attempts to generalize for unseen images reveal that the differences in underlying dynamics between CLIP and DDPM lead to less-than-perfect results. To continue exploring this issue, one could attempt guidance on a model that operates in a similar latent space, such as Stable Diffusion, which will be discussed in the next section.

III. AN INTRODUCTION TO STABLE DIFFUSION

This section focuses on the implementation and detailed analysis of the Stable Diffusion (SD) model using the CompVis/stable-diffusion-v1-4 configuration [16]. The SD model combines a Variational Autoencoder (VAE) and latent diffusion mechanisms, which allows it to generate images

efficiently by compressing the high-dimensional image data into a smaller latent space. The first step involves setting up the model and generating images based on textual prompts through a text encoder a CLIP tokenizer. Then we explore the deeper aspects of the architecture.

A. Image Generation Process

The model used is "CompVis/stable-diffusion-v1-4." The CLIP tokenizer and text encoder are loaded to process the input prompt into a format compatible with the diffusion model. The text is tokenized and encoded into a latent representation that the model uses to guide the image generation. The encoded text embeddings serve as the conditioning input, guiding the UNet component in generating an image that corresponds to the prompt. Once the model is initialized, the process involves passing a textual prompt (e.g., "A beautiful sunset above sea and mountains") through the tokenizer and the text encoder to produce text embeddings. These embeddings are used as input to the UNet model, which processes the latent noise to generate the image. Figure 8 illustrates an example output image generated using the above function.



Fig. 8: Image generated using the Stable Diffusion model conditioned on the prompt "A beautiful sunset above the sea."

B. Latent Diffusion and VAE Compression

After understanding the image generation setup, we delve into the latent diffusion process, which is crucial for handling high-resolution images. Latent diffusion mitigates computational costs by compressing the image into a latent space using a Variational Autoencoder (VAE). The VAE encodes the high-dimensional pixel space (x) into a lower-dimensional latent representation ($z = \mathcal{E}(x)$), which allows the model to perform diffusion in a computationally efficient manner. The process retains essential information while reducing the image dimensions significantly.

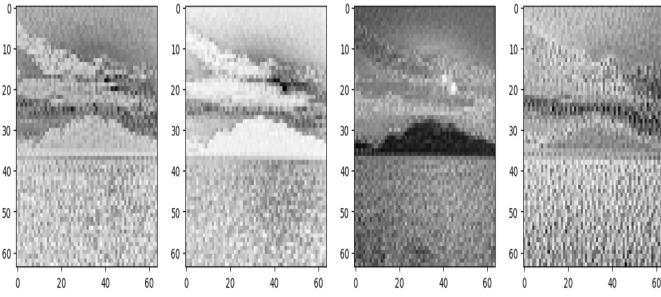


Fig. 9: The latent diffusion process in SD where high-resolution images are compressed into a smaller latent space for efficient processing.

Latent Diffusion Models (LDMs) use the VAE to encode the image into a latent space, reducing its spatial dimensions by a factor of $F = \frac{H}{h} = \frac{W}{w}$. This downscaling significantly reduces memory usage, allowing the model to generate high-resolution images efficiently. To avoid high-variance latent spaces, the model uses two types of regularizations: KL-regularization (KL-reg.) and vector quantization (VQ-reg.). KL-reg. applies a slight penalty to ensure the latent space follows a normal Gaussian distribution, which stabilizes the reconstructions. In contrast, VQ-reg. maps the latent representations to discrete values, improving stability and performance when integrated with the diffusion process.

C. Latent Space Compression and Efficiency

VAE compresses 512x512 pixel images into a latent space of dimensions 4x64x64, achieving a compression factor of 48. This compact representation reduces computational overhead while maintaining high-fidelity image reconstructions. The reduced dimensionality enables efficient processing during the diffusion steps, allowing the SD model to handle high-resolution images using manageable computational resources. The diagram in Figure 10 further illustrates the overall architecture of the latent diffusion process, detailing the integration of conditioning information like text and semantic maps via the cross-attention mechanism within the UNet.

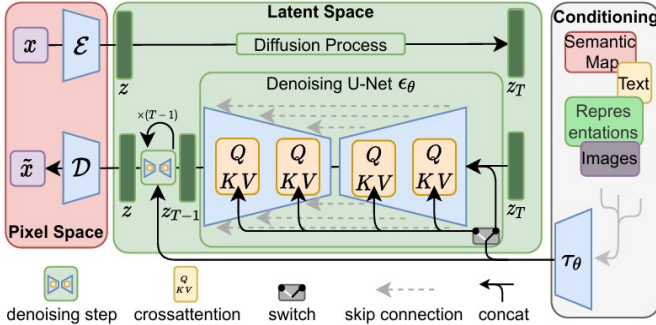


Fig. 10: Detailed diagram of the latent diffusion process in Stable Diffusion [8]

1) **Denoising and Cross-Attention Mechanism:** The denoising process in the latent space leverages a denoising UNet,

ϵ_θ , which iteratively refines the noisy latent representations, z_t , to recover the clean version. The UNet uses cross-attention layers to incorporate different types of conditioning inputs, such as text embeddings from CLIP or semantic maps, enhancing the fidelity of the generated images. The cross-attention mechanism, shown in the diagram, uses query (Q), key (K), and value (V) components to match relevant parts of the conditioning information with the latent representation. This approach ensures that each spatial location in the latent space is informed by the contextual information provided by the conditioning input, leading to more accurate and contextually relevant image generation.

2) **Optimization Objective:** The optimization objective for the latent diffusion model (LDM) is defined as:

$$L_{LDM} = E_{\mathcal{E}(x), \epsilon \sim \mathcal{N}(0,1), t} [\|\epsilon - \epsilon_\theta(z_t, t)\|^2] \quad (1)$$

Here, z_t represents the noisy latent at timestep t , and ϵ is noise sampled from a standard normal distribution. The model optimizes this objective by minimizing the difference between the predicted and actual noise, progressively refining the image through each denoising step.

D. Generative Modeling of Latent Representations

In the latent space, the generative process begins with a noisy latent z_T , which is gradually denoised back to a clean latent representation, z_0 , using the conditioned UNet. This approach allows the model to operate in a low-dimensional space, making high-resolution image synthesis feasible without excessive computational demands. The VAE ensures that the compressed latent representations retain essential details for accurate image reconstruction, while the latent diffusion mechanism adds robustness and flexibility, making the overall process efficient and effective for generating high-fidelity outputs.

By combining latent space compression with an efficient denoising process, Stable Diffusion achieves a balance between computational efficiency and high-quality image synthesis, suitable for generating complex, high-resolution images efficiently.

IV. TEXT CONDITIONING AND CLIP INTEGRATION: THEORETICAL ASPECTS

The Stable Diffusion (SD) model supports text conditioning through the CLIP text encoder, which transforms text prompts into embeddings that the UNet uses to guide the generation process. This conditioning mechanism provides the model with additional information about the image to be generated, allowing it to produce outputs that align closely with the text description provided.

A. Text Encoding Process

The CLIP model leverages a pre-trained transformer architecture designed to process image captions and compare images with text. When a text prompt is provided, CLIP tokenizes the input using a large vocabulary, assigning a specific token to each word. These tokens are then passed through

the CLIP text encoder, which generates a high-dimensional vector for each token. In Stable Diffusion v1.4 each token is converted into a 768-dimensional vector, and these vectors are padded or truncated to a consistent length of 77 tokens. The final representation, which is used as conditioning input is a tensor of shape (77×768) . This encoded representation (`encoder_hidden_states`) are then fed into the UNet as a conditioning input during denoising. The following figure shows the text encoding process in SD where the input tokenized, embedded and transformed through the stack of transformer blocks to generate embeddings.

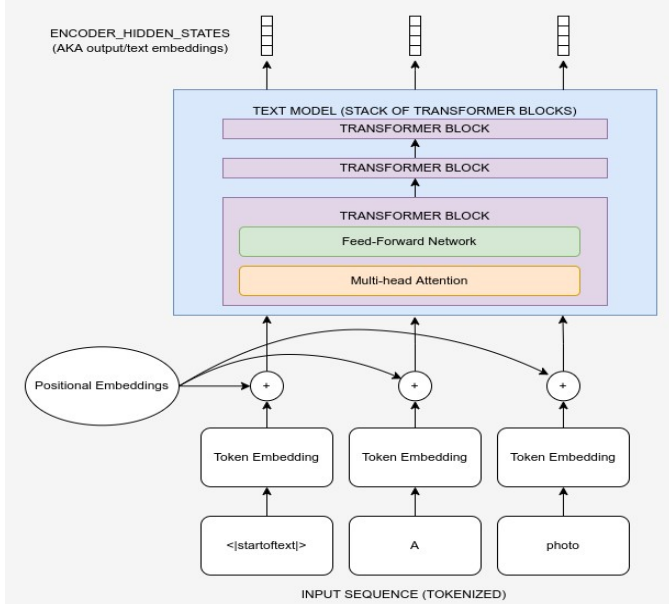


Fig. 11: CLIP text encoding process used in Stable Diffusion [9]

B. Cross-Attention Mechanism in the UNet

The encoded text embeddings generated by the CLIP model are incorporated into the UNet using cross-attention layers. These cross-attention mechanisms are scattered throughout the UNet’s architecture, allowing the model to attend to different parts of the text embeddings at various spatial locations. By doing so, the UNet aligns the noisy latent representations with the details from the prompt, ensuring that the generated image accurately reflects the textual description. This process works by matching each spatial location in the UNet with relevant tokens from the text embeddings. The cross-attention layers compute attention scores using query (Q), key (K), and value (V) vectors, bringing in information from the text prompt at each level of the UNet. As shown in Figure 12, the text conditioning information (along with timestep-based conditioning) is fed into different stages of the UNet, enhancing the coherence and quality of the generated image.

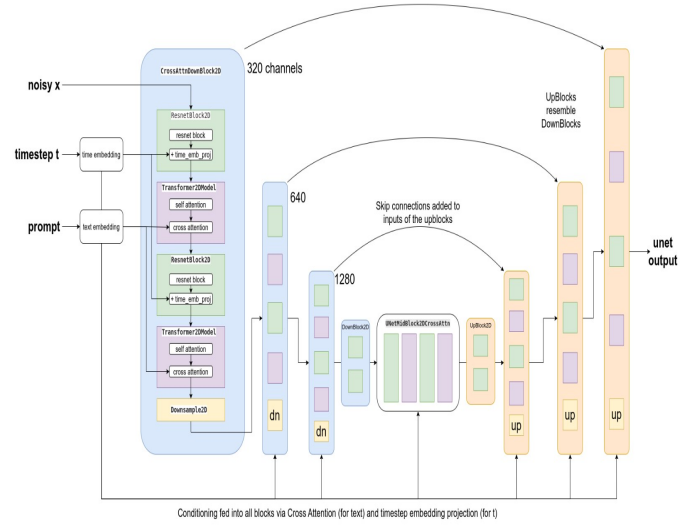


Fig. 12: Conditioning process in the UNet, where cross-attention layers integrate text embeddings and timestep information at various stages. This enables the model to generate images that accurately align with the input prompt [8].

By integrating text conditioning via cross-attention, Stable Diffusion allows for detailed and contextually relevant image synthesis, making use of the extensive information embedded in the text prompt at every level of the generation process.

V. EXPERIMENTS ON STABLE DIFUSSION

Various tests were conducted to evaluate the performance and sensitivity of the SD model to different parameters. These experiments explore the effects of guidance scale, noise scheduling, and latent space manipulation.

A. Classifier-Free Guidance (CFG) Analysis

CFG adjusts the model’s reliance on text conditioning, influencing the alignment between the generated image and the input prompt. We tested CFG with scales ranging from 1 to 12, observing how it affects the image quality. The results are shown in Figure 13.

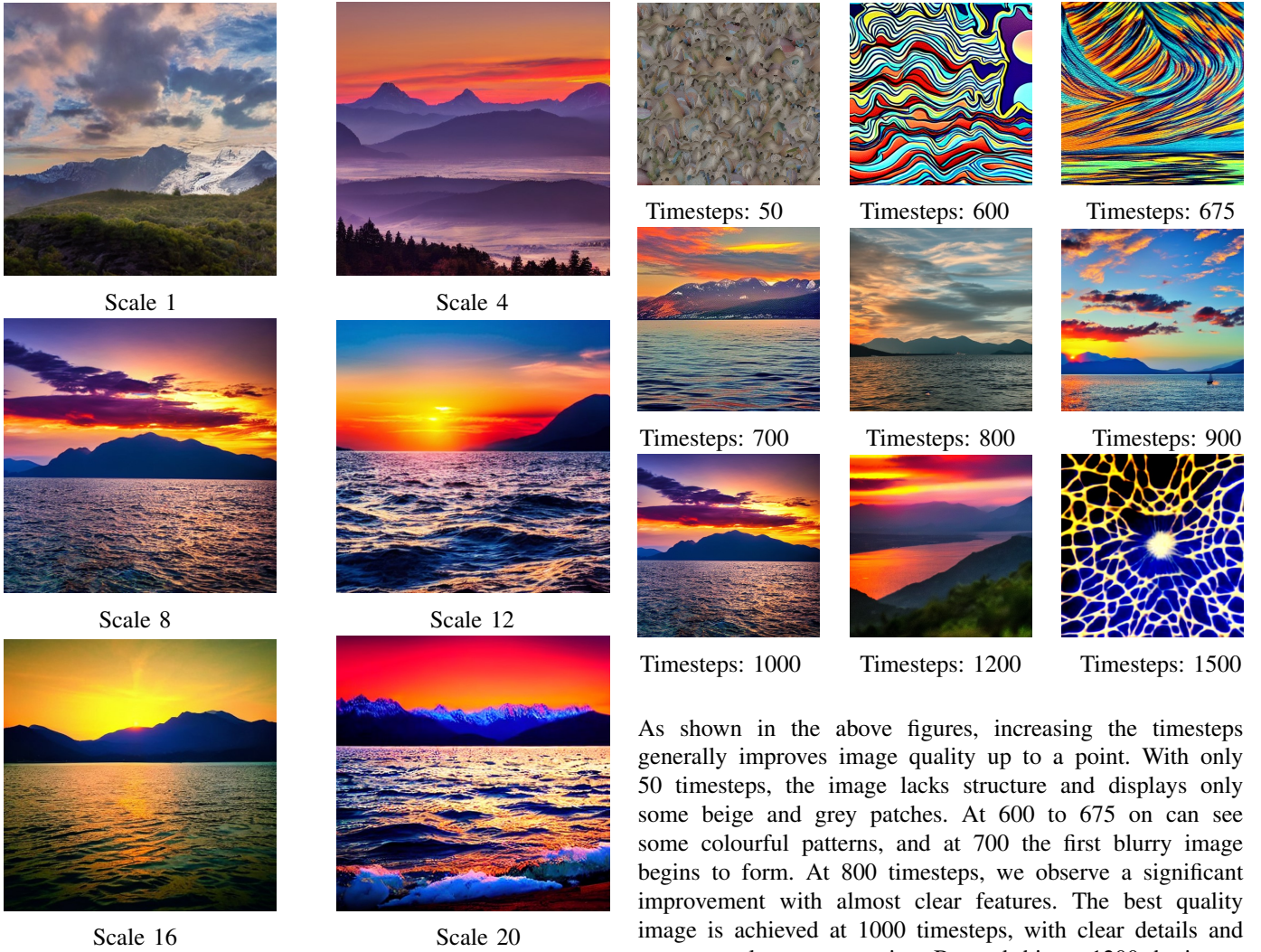


Fig. 13: Images generated with different CFG scales for the prompt “A sunset over the sea with mountains.”

As the CFG scale increases, the alignment between the generated image and the text prompt improves significantly. At lower scales (1 up to 5) the generated images lack detail and deviate from the specified scene. However, as the scale is increased to 8 and 16, the images are more clear, have more vibrant colors and better alignment with the prompt’s description, but also higher quality and more realistic outputs. However, this improvement comes at a cost, since higher CFG scales increase the computational load, resulting in much slower generation (scale = 12 took approximately 3 minutes for 1 image, running on an 8Gb Nvidia RTX3070 GPU). At 20 it is obvious that the image significantly loses quality.

B. Noise Scheduling Experiments

The noise schedule is managed by the LMSDiscreteScheduler. We tested variations in timesteps, ranging from 50 to 1500, to analyze the trade-off between image quality and generation time. The results shown in the following figures show how increasing the number of timesteps impacts the visual quality of the generated images.

As shown in the above figures, increasing the timesteps generally improves image quality up to a point. With only 50 timesteps, the image lacks structure and displays only some beige and grey patches. At 600 to 675 one can see some colourful patterns, and at 700 the first blurry image begins to form. At 800 timesteps, we observe a significant improvement with almost clear features. The best quality image is achieved at 1000 timesteps, with clear details and accurate color representation. Beyond this, at 1200 the image quality begins to slightly degrade, and by 1500 timesteps the image is over-processed and distorted.

C. Latent Space Manipulation

To understand the latent space’s structure and its impact on image reconstruction, we conducted several manipulations using the latent vectors. Starting with an image generated based on the prompt “A beautiful picture of a sport car in Tokyo,” we encoded it into the latent space and added various noise levels to observe how the output changes. The following figures show these steps sequentially:



Fig. 14: Generated image based on the prompt.

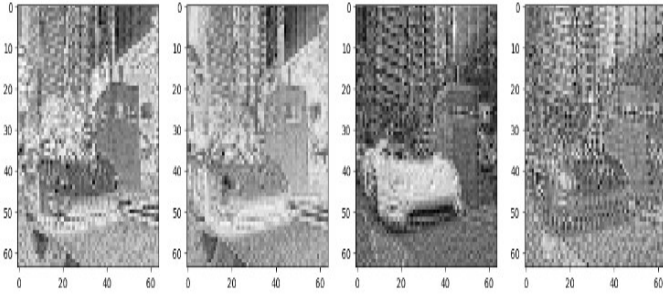


Fig. 15: Image encoded into latent space, showing its four-channel representation.



Fig. 16: Image decoded from latent space, demonstrating accurate reconstruction.

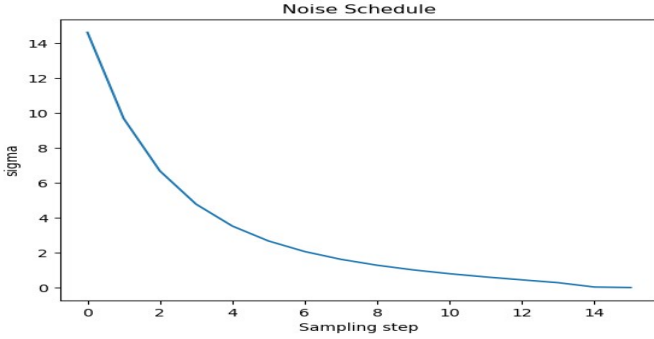


Fig. 17: Noise schedule applied using the LMSDiscreteScheduler, showing the reduction of sigma values over sampling steps.



Fig. 18: Image generated after adding noise to the latent vector and decoding it.

The results show that the encoding to latent space retains the structure and essence of the original image. When the

latent vectors are decoded without additional noise, the output closely matches the input. However, as seen in Figure 18, adding noise at specific sampling steps significantly distorts the image, emphasizing the importance of precise noise scheduling during the image generation process. The noise schedule (Figure 17) controls the amount of noise applied at each step, influencing the overall quality of the generated image.

VI. CLIP-GUIDED IMAGE GENERATION

The integration of CLIP guidance enhances control over image generation quality and adherence to prompts. CLIP guidance works by computing a similarity score between the text embeddings (from the prompt) and the visual features of the generated image. This score is used to adjust the latents in each iteration, aligning the image generation closer to the semantic content of the prompt.

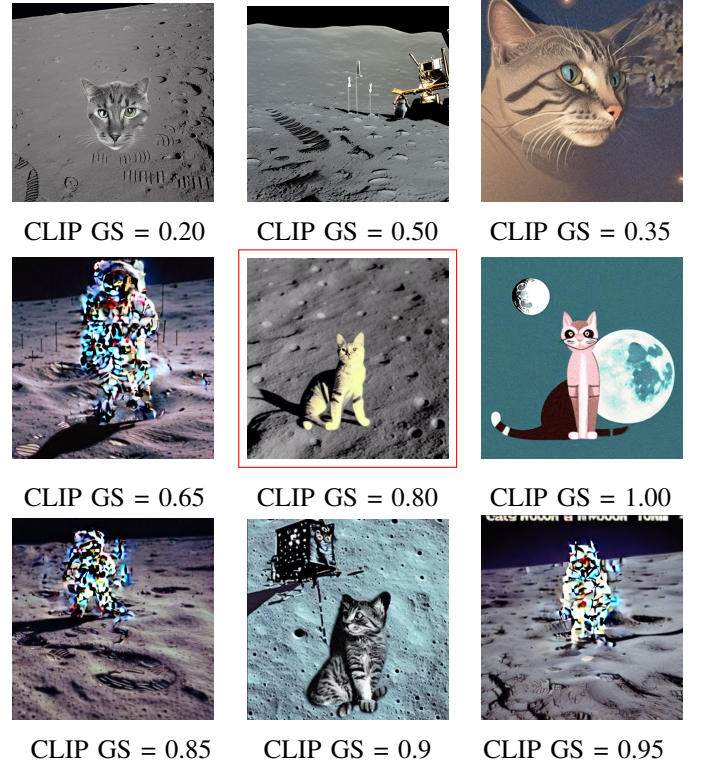


Fig. 19: Images generated with varying CLIP guidance strengths for the prompt “a cat on the moon.”

We conducted experiments using the prompt “a cat on the moon” and varied the CLIP guidance strength to evaluate the model’s response. The results, shown in Figure 19, demonstrate the impact of different CLIP guidance strengths. At low strengths (0.2 and 0.65), the generated images fail to accurately capture the prompt, often displaying abstract or random features. As the strength increases to 0.8, the image quality and alignment improve significantly, resulting in the best visual output that also better matches the prompt description. However, further increasing the strength (values between

0.8 and 1) results in distorted images, indicating that excessive CLIP guidance can over-penalize the generation, leading to instability. Therefore, a CLIP guidance strength of 0.8 was found to produce the best results, balancing prompt capture and image quality effectively.

VII. NOVEL IMAGE GENERATIONS

To explore the creative potential of the model, we used a variety of imaginative prompts and applied CLIP guidance based on our previous findings. Below are several novel image generations:



Figure N1



Figure N2



Figure N3



Figure N4

Fig. 20: Novel image generations using various imaginative prompts and CLIP guidance.

Figure N1: A spaceship docking at a crystal space station orbiting a giant blue planet.

Figure N2: A dancer made of smoke and light, performing on a stage of shattered glass.

Figure N3: Cleopatra exploring an ancient library filled with glowing hieroglyphs.

Figure N4: A floating library in the sky soaring through the clouds.

VIII. BERT-GUIDED IMAGE GENERATION

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based model designed primarily for natural language understanding tasks, such as text classification and named entity recognition. Unlike CLIP, which aligns image and text representations in a shared embedding space, BERT focuses on capturing the contextual relationships between words in a sentence using a bidirectional approach. This means that BERT processes the entire input sequence at

once, considering both left and right contexts simultaneously, providing a deep semantic understanding of the text.

When using BERT for image generation, we utilize the embeddings generated by the BERT model as a replacement for the traditional CLIP embeddings. The purpose is to evaluate how BERT embeddings, which are typically used for understanding textual information, influence the generation process when applied as guidance in the image synthesis pipeline.

In this experiment, we applied various guidance scales, ranging from low to high, to observe the model’s behavior under different BERT-guided conditions. The images were generated using the prompt “A futuristic cityscape under a neon sky.” Below, we present the generated outputs alongside their respective BERT guidance scales.



BERT GS = 0.01



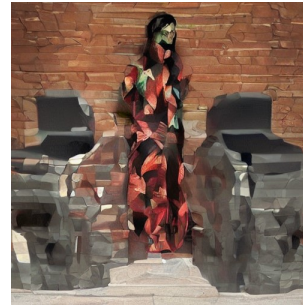
BERT GS = 2



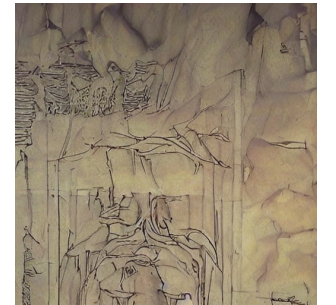
BERT GS = 4



BERT GS = 6



BERT GS = 8



BERT GS = 10

Fig. 21: Images generated with varying BERT guidance strengths for the prompt “A futuristic cityscape under a neon sky.”

As shown in Figure 21, the influence of BERT guidance on the generated images varies significantly with the guidance strength:

- **Low Guidance (0.01):** At the lowest guidance level, the generated image appears abstract and unstructured, as the influence of BERT embeddings is minimal. The resulting image does not accurately reflect the prompt and lacks meaningful features.
- **Moderate Guidance (2-6):** Increasing the guidance strength to values between 2 and 6 introduces more structure and patterning in the images. The images still do not capture the essence of the prompt and remain distorted and lack clarity.
- **High Guidance (8-10):** At high guidance values, the influence of BERT becomes dominant. However, rather than improving image clarity or alignment with the prompt, the images become chaotic, even though we can see some picture for GS=8, but still not a meaningful image. From that point on the higher the GS the more the images deviate into abstract shapes and textures that do not convey the intended neon-lit cityscape. This indicates that while BERT can provide some level of guidance, it still fails to generate meaningful images based on the input prompt, and this is due to its lack of visual understanding, and for that is mainly used for NLP tasks.

IX. CONCLUSION

Through our experimentation, we explored multiple approaches to guide image generation using CLIP and BERT embeddings. While DDPM serves as a reliable basic model that can be easily fine-tuned, the differences between the pixel space of images and the latent space of CLIP embeddings make it more challenging to generalize compared to Stable Diffusion. In the latter, CLIP embeddings, when adjusted with optimal guidance strengths, effectively aligned the generated images with their prompts. However, BERT embeddings displayed limitations due to their lack of multimodal training. As seen in the results, BERT-guided generations lacked coherence and failed to represent the intended visual elements of the prompts. This highlights the importance of using models trained specifically for aligning visual and textual information when guiding image generation. CLIP's ability to bridge the gap between text and image modalities provides significant advantages over traditional text-only models like BERT.

REFERENCES

- [1] **Generative Adversarial Nets:**
Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y. (2014).
https://proceedings.neurips.cc/paper_files/paper/2014/file/5ca3e9b122f61f8f06494c97b1afccf3-Paper.pdf
- [2] **Auto-Encoding Variational Bayes:**
Kingma, D. P., Welling, M. (2013).
<https://arxiv.org/pdf/1312.6114>
- [3] **Denoising Diffusion Probabilistic Models:**
Ho, J., Jain, A., Abbeel, P. (2020).
<https://arxiv.org/pdf/2006.11239>
- [4] **Deep Unsupervised Learning using Nonequilibrium Thermodynamics:**
Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., Ganguli, S. (2015).
<https://arxiv.org/pdf/1503.03585>
- [5] **Denoising Diffusion Implicit Models:**
Song, J., Meng, C., Ermon, S. (2020).
<https://arxiv.org/pdf/2010.02502>
- [6] **Butterfly Dataset:**
Veeralakrishna. (2021).
<https://www.kaggle.com/datasets/veeralakrishna/butterfly-dataset>
- [7] **WikiArt Dataset:**
Hugging Face. (2021).
<https://huggingface.co/datasets/huggan/wikiart>
- [8] **High-Resolution Image Synthesis with Latent Diffusion Models:**
<https://arxiv.org/pdf/2112.10752>
- [9] **Stable Diffusion:**
<https://huggingface.co/learn/diffusion-course/en/unit3/1>
- [10] **Fine-Tuning and Guidance:**
<https://huggingface.co/learn/diffusion-course/en/unit2/2what-you-will-learn>
- [11] **Training with Diffusers:**
https://colab.research.google.com/github/huggingface/notebooks/blob/main/diffusers/training_example.ipynb
- [12] **Learning Transferable Visual Models From NL Supervision (CLIP):**
<https://arxiv.org/pdf/2103.00020>
- [13] **CLIP API and Example Usage:**
<https://github.com/OpenAI/CLIP>
- [14] **Accelerate Inference of Text-to-Image Diffusion Models:**
https://huggingface.co/docs/diffusers/main/en/tutorials/fast_diffusion
- [15] **Diffusion Models for Image Generation – Guide:**
<https://learnopencv.com/image-generation-using-diffusion-models/>
- [16] **stable-diffusion-v1-4 from Hugging face:**
<https://huggingface.co/CompVis/stable-diffusion-v1-4>